

API Integration & Authentication Guide

Enterprise Resolution Intelligence System API v3.5

Base URL:	https://api.example.com/v3	Auth:	API Key / OAuth 2.0
-----------	----------------------------	-------	---------------------

1. Authentication Methods

Method	Use Case	Implementation
API Key	Server-to-server, backend services	Header: X-API-Key: your_key
OAuth 2.0	User-delegated access, web/mobile apps	Authorization Code / Client Credentials
JWT Bearer	Stateless authentication, microservices	Header: Authorization: Bearer <token>

1.1 API Key Management

API keys provide simple authentication for server-to-server integrations. Keys are scoped to specific permissions and can be rotated.

Generate API Key:

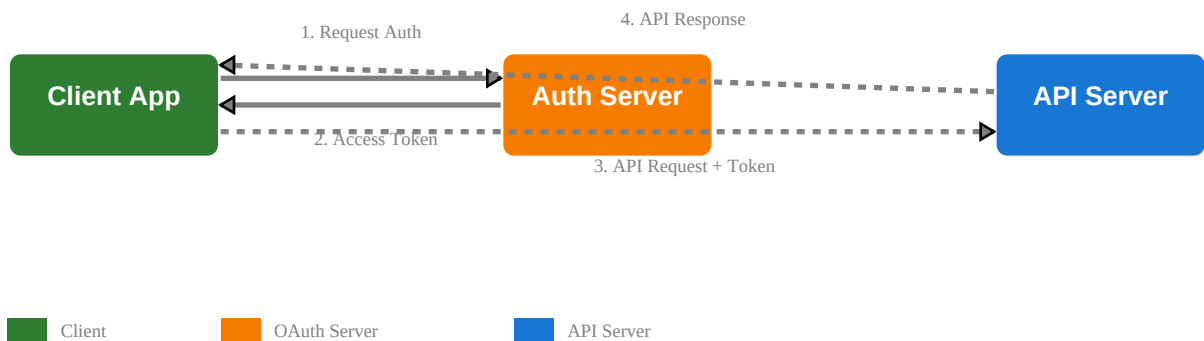
```
POST /v3/auth/api-keys
{ "name": "Production Key", "scopes": ["read:tickets", "write:tickets"] }
```

Use API Key in Requests:

```
curl -H "X-API-Key: sk_live_abc123xyz" https://api.example.com/v3/tickets
```

- Keys prefixed with `sk_live_` (production) or `sk_test_` (sandbox)
- Rotate keys every 90 days or immediately if compromised
- Store keys securely in environment variables or secrets manager

1.2 OAuth 2.0 Authorization Flow



Step 1: Redirect User to Authorization URL

```
GET https://auth.example.com/oauth/authorize?
client_id=your_client_id&response_type=code&redirect_uri=your_callback
```

Step 2: Exchange Authorization Code for Access Token

```
POST https://auth.example.com/oauth/token
{ "grant_type": "authorization_code", "code": "auth_code", "client_id": "...", "client_secret": "..." }
```

Token Response:

```
{ "access_token": "eyJhbG...", "token_type": "Bearer", "expires_in": 3600, "refresh_token": "..." }
```

2. Rate Limiting Rules

Tier	Requests/Min	Daily Quota	Burst Limit
Free	60	10,000	100/10s
Basic	300	100,000	500/10s
Professional	1,200	500,000	2,000/10s
Enterprise	Custom	Unlimited	Custom

Rate Limit Headers:

- X-RateLimit-Limit: Maximum requests allowed
- X-RateLimit-Remaining: Requests remaining in current window
- X-RateLimit-Reset: Unix timestamp when limit resets

Handling Rate Limit Exceeded (429):

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
{ "error": "rate_limit_exceeded", "message": "Rate limit exceeded. Retry after 60 seconds" }
```

3. API Error Codes Reference

Code	Error	Description	Resolution
400	Bad Request	Invalid request format or parameters	Check request body and parameters
401	Unauthorized	Missing or invalid authentication	Provide valid API key or token
403	Forbidden	Insufficient permissions for resource	Check API key scopes
404	Not Found	Resource does not exist	Verify resource ID
409	Conflict	Resource state conflict	Retry with updated data
422	Validation Error	Request validation failed	Fix validation errors in response
429	Rate Limited	Too many requests	Implement exponential backoff
500	Internal Error	Server-side error	Retry or contact support
502	Bad Gateway	Upstream service unavailable	Retry with exponential backoff
503	Service Unavailable	Temporary service outage	Check status page, retry later

3.1 Standard Error Response Format

```
{
  "error": { "code": "validation_error", "message": "Invalid email format",
  "details": [{ "field": "email", "issue": "Must be valid email" }],
  "request_id": "req_abc123" }
}
```

4. Integration Best Practices

4.1 Security

- Never expose API keys in client-side code or version control
- Use HTTPS for all API requests
- Implement token refresh logic for OAuth flows
- Rotate API keys periodically (recommended: every 90 days)

4.2 Error Handling

- Always check HTTP status codes before parsing response
- Implement retry logic with exponential backoff for 5xx errors
- Log request_id from error responses for support tickets
- Handle rate limiting gracefully using Retry-After header

4.3 Performance

- Use pagination for list endpoints (max 100 items per page)
 - Implement request caching to reduce API calls
 - Use webhooks for real-time updates instead of polling
 - Batch requests when possible to optimize rate limit usage
-

5. Quick Start Code Examples

Python Example:

```
import requests

headers = {"X-API-Key": "sk_live_abc123", "Content-Type": "application/json"}

response = requests.get("https://api.example.com/v3/tickets", headers=headers)

if response.status_code == 200: print(response.json())
```

JavaScript Example:

```
const response = await fetch('https://api.example.com/v3/tickets', {
  headers: { 'X-API-Key': 'sk_live_abc123', 'Content-Type': 'application/json' }
});

const data = await response.json();
```

cURL Example:

```
curl -X GET https://api.example.com/v3/tickets \
-H "X-API-Key: sk_live_abc123" \
-H "Content-Type: application/json"
```

6. Key API Endpoints

Endpoint	Method	Description
/v3/tickets	GET	List all support tickets
/v3/tickets	POST	Create new ticket
/v3/tickets/:id	GET	Get ticket details
/v3/tickets/:id	PATCH	Update ticket
/v3/users	GET	List users
/v3/analytics	GET	Get analytics data
/v3/webhooks	POST	Register webhook

API Support Resources

Documentation: <https://docs.example.com/api> | Developer Portal: <https://developers.example.com>
API Status: <https://status.example.com> | Support: api-support@example.com